

Sistem de management al riscului bancar și alocării creditelor

Facultatea de Matematică și Informatică

Sisteme si Gestiunea Bazelor de Date

Rachieriu Gheorghe Gabriel

Seria: 24

Grupa: 241

An universitar: 2025-2026

Contents

Prefață	3
1 Descrierea modelului real	3
2 Diagrama Entități - Relații ERD	4
2.1 Entități și Atribute	4
2.2 Relații între entități	4
2.3 Relația Ternară EVALUARE	4
2.4 Diagrama Entități - Relații	5
3 Diagrama Conceptuală	6
4 Creare Tabele	7
4.1 Tabela VENIT	7
4.2 Tabela CLIENT	7
4.3 Tabela ANALIST	7
4.4 Tabela ACHIZITIE	7
4.5 Tabela VENIT_CLIENT	8
4.6 Tabela GARANT	8
4.7 Tabela CREDIT	8
4.8 Tabela EVALUARE	9
4.9 Tabela RATA	9
4.10 Tabela PLATA	10
4.11 Trigger Generare Rate	10
5 Introducere Date	11
6 Subprogram stocat independent	11
6.1 Enunț	11
6.2 Rezolvare	11
7 Cursoare - Subprogram Stocat Independent	15
7.1 Enunț	15
7.2 Rezolvare	15
8 NO_DATA_FOUND si TOO_MANY_ROWS - Subprogram Stocat Independent	18
8.1 Enunț	18
8.2 Rezolvare	18
9 Subprogram Stocat Independent Procedură - 2 Excepții Proprii	20
9.1 Enunț	20
9.2 Rezolvare	20
10 Trigger LMD	23
10.1 Enunț	23
10.2 Rezolvare	23
11 Trigger LMD la nivel de linie	25
11.1 Enunț	25
11.2 Rezolvare	25
12 Trigger LDD	27
12.1 Enunț	27
12.2 Rezolvare	27

13 Pachet: Flux Actiuni Integrate	29
13.1 Enunt	29
13.2 Rezolvare	29

Prefață

Pentru implementarea modelului de bază de date am configurat pe un sistem de operare **macOS cu Apple Silicon**. Instanța de baze de date **Oracle Database 19c** a fost rulată într-un **container Docker**. Interogările SQL, definirea schemelor și testarea funcționalităților au fost realizate folosind **JetBrains DataGrip**.

1 Descrierea modelului real

Acest proiect modelează baza de date a unei bănci, având ca scop gestionarea creditelor și a riscurilor asociate acestora. Sistemul verifica tot procesul: adaugarea unui client nou, creditului și plata ratelor lunare.

Modelul este construit pe baza următoarelor reguli:

Clienți și Venituri

Baza de date stochează informațiile personale ale **clienților** (nume, CNP, adresă) și situația lor financiară. Pentru a calcula corect dacă un client își permite un credit, am modelat o relație prin care unui client i se pot asocia mai multe tipuri de **venituri** (salarii, chirii, etc.). Un aspect important este *gradul de îndatorare*, care nu trebuie să depășească o anumită limită (40%) pentru aprobarea dosarului.

Garanți

Pentru creditele mai mari, banca cere **garanți**. În modelul real, nu contează doar venitul garantului, ci și relația acestuia cu clientul. Baza de date permite analiza acestor legături pentru a vedea dacă garantul este rudă (soț, părinte, copil) sau o persoană independentă, acest lucru influențând riscul dosarului.

Acordarea Creditelor și Achiziții

Un client poate solicita unul sau mai multe **credite**. De cele mai multe ori, creditul este legat de o **achiziție** specifică (de exemplu, un apartament). Am inclus tabela de achiziții pentru a putea compara valoarea de piață a bunului cu valoarea evaluată de bancă, asigurând astfel că suma împrumutată este echitabilă ambelor părți.

Evaluarea Riscului

Partea principală a sistemului este **evaluarea**. Un **analist** bancar verifică dosarul format din cererea de credit și achiziția propusă. În urma analizei, se stabilește un *scor de risc* și o *clasă de risc* (A, B sau C). Decizia finală de aprobare se bazează pe aceste date.

Rate și Plăți

Odată aprobat creditul, acesta este împărțit automat într-un număr de **rate**. Baza de date ține evidența scadențelor și a **plăților** efectuate. Dacă o rată nu este plătită la data scadență, sistemul permite calcularea automată a penalizărilor și actualizarea datoriilor restante.

2 Diagrama Entități - Relații ERD

2.1 Entități și Attribute

- CLIENT: id_persoana (PK), nume, prenume, data_nastere, adresa, telefon, email, CNP, venit_lunar, status_financiar, status_angajare, grad_indatorare.
- GARANT: id_GARANT (PK), nume, prenume, data_nastere, adresa, telefon, email, CNP, relatie_client, venit_garant.
- ANALIST: id_ANALIST (PK), nume, prenume, data_nastere, adresa, telefon, email, CNP, departament, suma_maxima.
- VENIT: id_venit (PK), id_persoana (FK), suma_lunara, tip_venit, sursa_venit, contract_venit
- ACHIZITIE: achizitie_id (PK), tip_achizitie, procent_finantat, suma_avans
- PLATA: id_plata (PK), id_rata (FK), data_plata, suma_platita, metoda_plata
- RATA: rata_id (PK), credit_id (FK), numar_rata, data_scadenta, indice_referinta, marja, suma_rata, suma_dobanda, status_rata, penalizari
- CREDIT: id_credit (PK), id_persoana (FK), tip_credit, suma_solicitata, suma_aprobata, data_aplicarii, data_acordarii, numar_rate
- EVALUARE: evaluare_id (PK), credit_id (FK), analist_id (FK), achizitie_id (FK), data_evaluarii, tip_evaluare, scor_risc, clasa_risc, decizie, comentarii

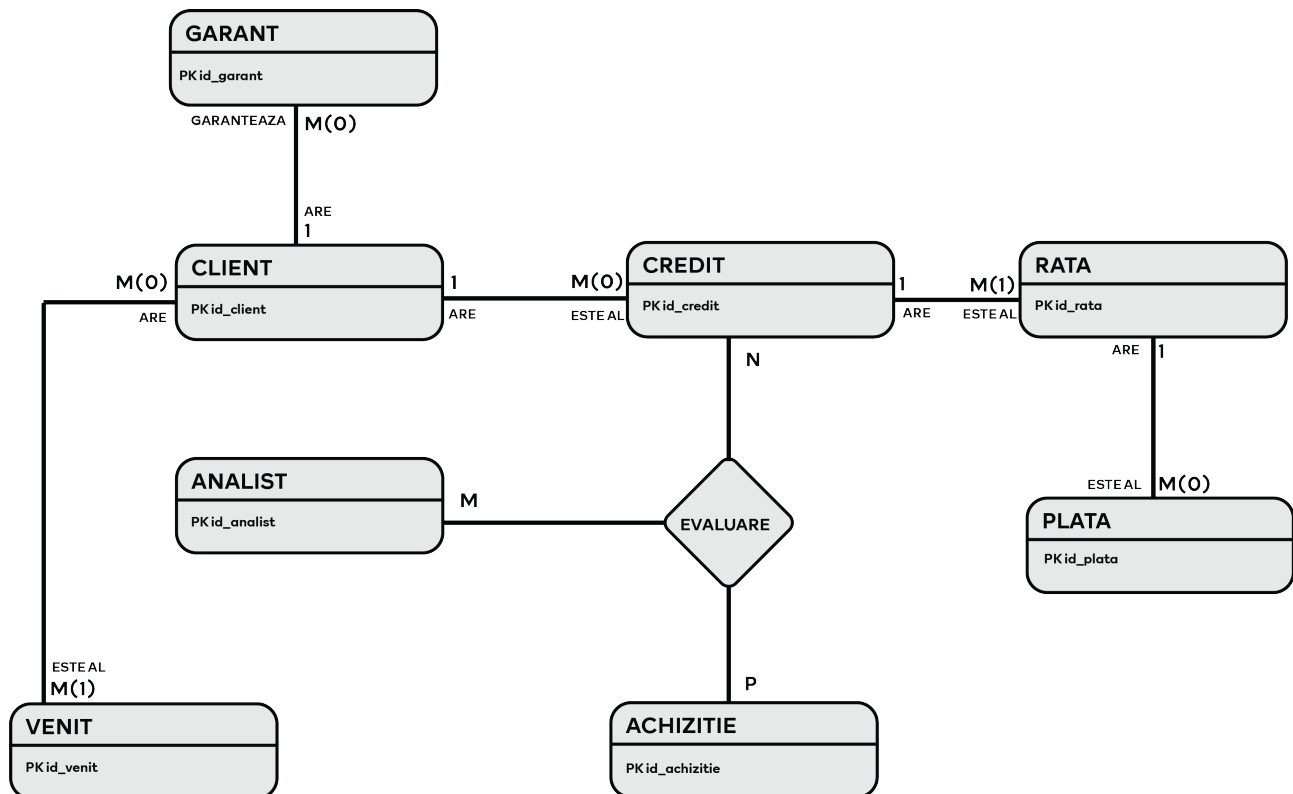
2.2 Relații între entități

- Un CLIENT are mai mulți GARANȚI sau niciunul. - **1(1) : M(0)**
- Un CLIENT are mai multe CREDITE sau niciunul. - **1(1) : M(0)**
- Mai multe VENITURI pot fi ale mai multor CLIENTI sau al niciunuia. - **M(1) : M(0)**
- Un CREDIT are cel puțin o RATA. - **1(1) : M(1)**
- O RATA are mai multe PLATI sau niciuna. - **1(1) : M(0)**

2.3 Relația Ternară EVALUARE

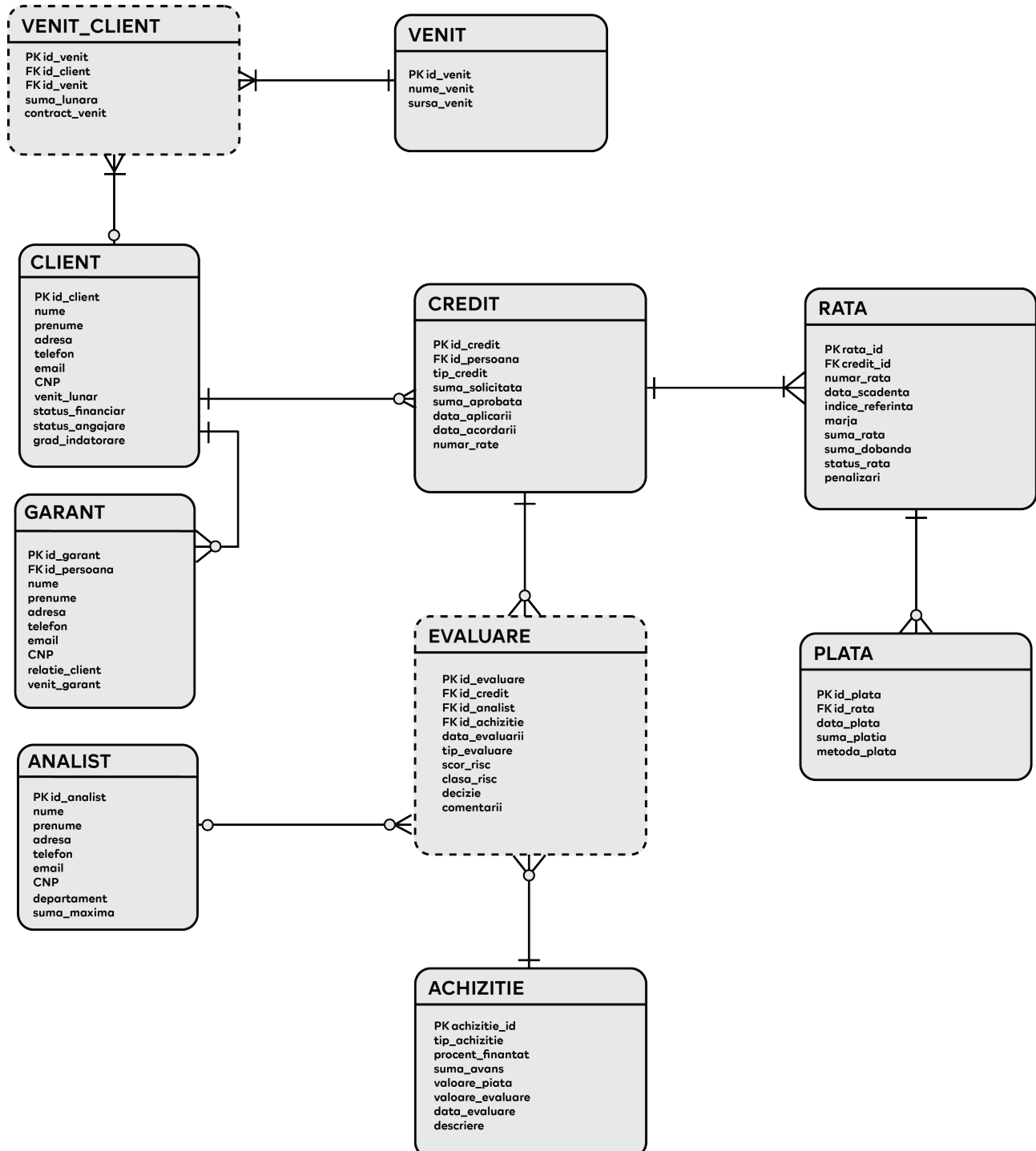
Relația EVALUARE este utilizată pentru a arăta procesul prin care banca analizează riscul unui credit. Fiecare evaluare este realizată de un analist asupra unei anumite achiziții propuse unui anumit credit, iar rezultatul acesteia include indicatori de risc, scoruri financiare și decizia finală de aprobare sau respingere. Deoarece evaluarea depinde simultan de toate cele trei entități (același credit poate avea evaluări multiple pentru achiziții diferite, un analist poate evalua mai multe credite, iar o achiziție poate fi evaluată de mai multe ori) relația nu poate fi descompusă în legături binare.

2.4 Diagrama Entități - Relații



3 Diagrama Conceptuală

Pentru rezolvarea relației many to many dintre CLIENT și VENIT, vom folosi tabelul asociativ **VENIT_CLIENT** cu atributele id_venit (PK), id_client (FK), id_venit (FK), suma_lunara si contract_venit.



4 Creare Tabele

4.1 Tabela VENIT

```
CREATE TABLE VENIT (  
    id_venit NUMBER GENERATED ALWAYS AS IDENTITY,  
    nume_venit VARCHAR2(100) NOT NULL,  
    sursa_venit VARCHAR2(100),  
  
    CONSTRAINT pk_venit PRIMARY KEY (id_venit)  
);
```

4.2 Tabela CLIENT

```
CREATE TABLE CLIENT (  
    id_client NUMBER GENERATED ALWAYS AS IDENTITY,  
    nume VARCHAR2(50) NOT NULL,  
    prenume VARCHAR2(50) NOT NULL,  
    adresa VARCHAR2(200),  
    telefon VARCHAR2(20),  
    email VARCHAR2(100),  
    CNP VARCHAR2(13) UNIQUE NOT NULL,  
    venit_lunar NUMBER(10,2),  
    status_financiar VARCHAR2(50),  
    status_angajare VARCHAR2(50),  
    grad_indatorare NUMBER(5,2),  
  
    CONSTRAINT pk_client PRIMARY KEY (id_client)  
);
```

4.3 Tabela ANALIST

```
CREATE TABLE ANALIST (  
    id_analist NUMBER GENERATED ALWAYS AS IDENTITY,  
    nume VARCHAR2(50) NOT NULL,  
    prenume VARCHAR2(50) NOT NULL,  
    adresa VARCHAR2(200),  
    telefon VARCHAR2(20),  
    email VARCHAR2(100),  
    CNP VARCHAR2(13),  
    departament VARCHAR2(50),  
    suma_maxima NUMBER(12,2),  
  
    CONSTRAINT pk_analist PRIMARY KEY (id_analist)  
);
```

4.4 Tabela ACHIZITIE

```
CREATE TABLE ACHIZITIE (  
    id_achizitie NUMBER GENERATED ALWAYS AS IDENTITY,  
    tip_achizitie VARCHAR2(50),  
    procent_finantat NUMBER(5,2),  
    suma_avans NUMBER(12,2),  
    valoare_piata NUMBER(12,2),  
    valoare_evaluare NUMBER(12,2),  
    data_evaluare DATE,  
    descriere VARCHAR2(255),  
  
    CONSTRAINT pk_achizitie PRIMARY KEY (id_achizitie)
```

```
);
```

4.5 Tabela VENIT_CLIENT

```
CREATE TABLE VENIT_CLIENT (  
    id_venit_client NUMBER GENERATED ALWAYS AS IDENTITY,  
    id_client NUMBER NOT NULL,  
    id_venit NUMBER NOT NULL,  
    suma_lunara NUMBER(10,2) NOT NULL,  
    contract_venit VARCHAR2(50),  
  
    CONSTRAINT pk_venit_client PRIMARY KEY (id_venit_client),  
  
    CONSTRAINT fk_vc_client FOREIGN KEY (id_client)  
    REFERENCES CLIENT(id_client)  
    ON DELETE CASCADE,  
  
    CONSTRAINT fk_vc_venit FOREIGN KEY (id_venit)  
    REFERENCES VENIT(id_venit)  
    ON DELETE CASCADE,  
  
    CONSTRAINT uq_client_venit UNIQUE (id_client, id_venit)  
);
```

4.6 Tabela GARANT

```
CREATE TABLE GARANT (  
    id_garant NUMBER GENERATED ALWAYS AS IDENTITY,  
    id_client NUMBER NOT NULL,  
    nume VARCHAR2(50),  
    prenume VARCHAR2(50),  
    adresa VARCHAR2(200),  
    telefon VARCHAR2(20),  
    email VARCHAR2(100),  
    CNP VARCHAR2(13),  
    relatie_client VARCHAR2(50),  
    venit_garant NUMBER(10,2),  
  
    CONSTRAINT pk_garant PRIMARY KEY (id_garant),  
  
    CONSTRAINT fk_garant_client FOREIGN KEY (id_client)  
    REFERENCES CLIENT(id_client)  
    ON DELETE CASCADE  
);
```

4.7 Tabela CREDIT

```
CREATE TABLE CREDIT (  
    id_credit NUMBER GENERATED ALWAYS AS IDENTITY,  
    id_client NUMBER NOT NULL,  
    tip_credit VARCHAR2(50) NOT NULL,  
    suma_solicitata NUMBER(12,2),  
    suma_aprobata NUMBER(12,2),  
    data_aplicarii DATE DEFAULT SYSDATE NOT NULL,  
    data_acordarii DATE,  
    numar_rate NUMBER(3) NOT NULL CHECK (numar_rate >= 1),  
  
    CONSTRAINT pk_credit PRIMARY KEY (id_credit),
```

```
CONSTRAINT fk_credit_client FOREIGN KEY (id_client)
REFERENCES CLIENT(id_client)
ON DELETE CASCADE
);
```

4.8 Tabela EVALUARE

```
CREATE TABLE EVALUARE (
    id_evaluare NUMBER GENERATED ALWAYS AS IDENTITY,
    id_credit NUMBER NOT NULL,
    id_analist NUMBER NOT NULL,
    id_achizitie NUMBER NOT NULL,
    data_evaluarii DATE DEFAULT SYSDATE NOT NULL,
    tip_evaluare VARCHAR2(50),
    scor_risc NUMBER(5,2),
    clasa_risc VARCHAR2(10),
    decizie VARCHAR2(50) NOT NULL,
    comentarii VARCHAR2(400),

    CONSTRAINT pk_evaluare PRIMARY KEY (id_evaluare),

    CONSTRAINT fk_eval_credit FOREIGN KEY (id_credit)
    REFERENCES CREDIT(id_credit)
    ON DELETE CASCADE,

    CONSTRAINT fk_eval_analist FOREIGN KEY (id_analist)
    REFERENCES ANALIST(id_analist)
    ON DELETE CASCADE,

    CONSTRAINT fk_eval_achizitie FOREIGN KEY (id_achizitie)
    REFERENCES ACHIZITIE(id_achizitie)
    ON DELETE CASCADE
);
```

4.9 Tabela RATA

```
REATE TABLE RATA (
    id_rata NUMBER GENERATED ALWAYS AS IDENTITY,
    id_credit NUMBER NOT NULL,
    numar_rata NUMBER(3) NOT NULL,
    data_scadenta DATE NOT NULL,
    indice_referinta VARCHAR2(50),
    marja NUMBER(5,2),
    suma_rata NUMBER(12,2) NOT NULL,
    suma_dobanda NUMBER(12,2),
    status_rata VARCHAR2(20),
    penalizari NUMBER(12,2) DEFAULT 0,

    CONSTRAINT pk_rata PRIMARY KEY (id_rata),

    CONSTRAINT fk_rata_credit FOREIGN KEY (id_credit)
    REFERENCES CREDIT(id_credit)
    ON DELETE CASCADE,

    CONSTRAINT uq_credit_rata UNIQUE (id_credit, numar_rata)
);
```

4.10 Tabela PLATA

```
CREATE TABLE PLATA (  
    id_plata NUMBER GENERATED ALWAYS AS IDENTITY,  
    id_rata NUMBER NOT NULL,  
    data_plata DATE DEFAULT SYSDATE,  
    suma_platita NUMBER(12,2) NOT NULL,  
    metoda_plata VARCHAR2(50),  
  
    CONSTRAINT pk_plata PRIMARY KEY (id_plata),  
  
    CONSTRAINT fk_plata_rata FOREIGN KEY (id_rata)  
    REFERENCES RATA(id_rata)  
    ON DELETE CASCADE  
);
```

4.11 Trigger Generare Rate

Deoarece conceptual, un credit este considerat valid dacă are cel puțin o rată, vom implementa un trigger care insereaza CREDIT.numar_rate de intrări în tabela RATA, astfel rezolvând problema cardinalității 1 : M(1) dintre CREDIT și RATA.

```
CREATE OR REPLACE TRIGGER trg_creare_rate  
AFTER INSERT ON CREDIT  
FOR EACH ROW  
BEGIN  
    FOR i IN 1 .. :NEW.numar_rate LOOP  
        INSERT INTO RATA ( id_credit, numar_rata, data_scadenta,  
            suma_rata, status_rata, penalizari) VALUES (  
            :NEW.id_credit,  
            i,  
            ADD_MONTHS(:NEW.data_acordarii, i),  
            (:NEW.suma_aprobata / :NEW.numar_rate),  
            'NEPLATITA',  
            0  
        );  
    END LOOP;  
END;  
/
```

5 Introducere Date

Deoarece script-ul de insert este voluminos am decis să nu îl introduc în documentație, însă acesta poate fi consultat în repository-ul de GitHub al proiectului. [Aici!](#)

6 Subprogram stocat independent

6.1 Enunț

Pentru un client identificat prin **id_client**, banca dorește să realizeze o analiză detaliată a situației financiare și a istoricului de creditare al acestuia. În acest scop, se cere implementarea unui **subprogram stocat independent** care utilizează toate cele trei tipuri de colecții studiate în PL/SQL: **nested table**, **varray** și **index-by table**.

Subprogramul trebuie să calculeze următoarele:

1. Analiza veniturilor clientului (NESTED TABLE)

Se preiau toate înregistrările corespunzătoare clientului din tabela **VENIT_CLIENT** într-o colecție de tip **nested table**. Pe baza acestei colecții se determină:

- venitul lunar total al clientului;
- lista tipurilor de venit ale clientului, fără valori duplicate;
- valoarea maximă a unui venit lunar individual.

2. Analiza creditelor aprobate (VARRAY)

Se preiau toate creditele aprobate ale clientului din tabela **CREDIT**, confirmate prin decizii favorabile din tabela **EVALUARE**, într-o colecție de tip **varray** cu o dimensiune maximă prestabilită (de exemplu, maximum 10 credite). Din această colecție se determină:

- numărul creditelor aprobate ale clientului;
- suma totală a valorilor aprobate pentru aceste credite.

3. Analiza penalizărilor ratelor (INDEX-BY TABLE + VARRAY)

Penalizările aferente ratelor creditelor clientului sunt preluate din tabela **RATA** într-o structură compusă, în care:

- cheia unui **index-by table** este anul scadenței ratei;
- valoarea este un **varray(12)** ce conține penalizările lunare pentru fiecare lună a anului.

Se cere:

- numărul total de penalizări ale clientului;
- suma totală a penalizărilor;
- afișarea penalizărilor lunare pentru anii primiți ca parametru subprogramului, sub forma unei colecții de tip **nested table**.

6.2 Rezolvare

```
CREATE OR REPLACE TYPE ani_nt AS TABLE OF NUMBER;
/

CREATE OR REPLACE PROCEDURE analiza_client(p_id_client NUMBER, p_anii ani_nt ) AS
    TYPE venit_nt IS TABLE OF VENIT_CLIENT%ROWTYPE;
    venituri venit_nt;

    TYPE tipuri_t IS TABLE OF BOOLEAN INDEX BY VARCHAR2(100);
    tipuri tipuri_t;

    venit_total NUMBER := 0;
    venit_maxim NUMBER := 0;
```

```
v_tip VARCHAR2(100);

TYPE credit_v IS VARRAY(10) OF CREDIT%ROWTYPE;
credite credit_v;
suma_totala NUMBER := 0;

TYPE luni_var IS VARRAY(12) OF NUMBER;
TYPE ani_tab IS TABLE OF luni_var INDEX BY PLS_INTEGER;
penalizari_ani ani_tab;

total_penalizari NUMBER := 0;
numar_penalizari NUMBER := 0;
temp luni_var;
BEGIN
    SELECT *
    BULK COLLECT INTO venituri
    FROM VENIT_CLIENT
    WHERE id_client = p_id_client;

    FOR i IN 1 .. venituri.COUNT LOOP
        venit_total := venit_total + venituri(i).suma_lunara;
        IF venituri(i).suma_lunara > venit_maxim THEN
            venit_maxim := venituri(i).suma_lunara;
        END IF;
    END LOOP;

    FOR i IN 1 .. venituri.COUNT LOOP
        SELECT v.nume_venit
        INTO v_tip
        FROM VENIT v
        WHERE v.id_venit = venituri(i).id_venit;

        IF NOT tipuri.EXISTS(v_tip) THEN
            tipuri(v_tip) := TRUE;
        END IF;
    END LOOP;

    DBMS_OUTPUT.PUT_LINE('Venit total: ' || venit_total);
    DBMS_OUTPUT.PUT_LINE('Venit maxim: ' || venit_maxim);
    DBMS_OUTPUT.PUT_LINE('Tipuri de venit:');

    v_tip := tipuri.FIRST;
    WHILE v_tip IS NOT NULL LOOP
        DBMS_OUTPUT.PUT_LINE(v_tip);
        v_tip := tipuri.NEXT(v_tip);
    END LOOP;

    SELECT DISTINCT c.*
    BULK COLLECT INTO credite
    FROM CREDIT c
    JOIN EVALUARE e ON c.id_credit = e.id_credit
    WHERE c.id_client = p_id_client
        AND LOWER(e.decizie) = 'aprobat';

    FOR i IN 1 .. credite.COUNT LOOP
        suma_totala := suma_totala + credite(i).suma_aprobata;
    END LOOP;
```

```

DBMS_OUTPUT.PUT_LINE('Numar credite aprobate: ' || credite.COUNT);
DBMS_OUTPUT.PUT_LINE('Suma totala aprobata: ' || suma_totala);

FOR r IN (
    SELECT r.penalizari, r.data_scadenta
    FROM RATA r
    JOIN CREDIT c ON r.id_credit = c.id_credit
    WHERE c.id_client = p_id_client
) LOOP
    DECLARE
        an NUMBER := EXTRACT(YEAR FROM r.data_scadenta);
        luna NUMBER := EXTRACT(MONTH FROM r.data_scadenta);
    BEGIN
        IF NOT penalizari_ani.EXISTS(an) THEN
            penalizari_ani(an) := luni_var(0,0,0,0,0,0,0,0,0,0,0,0);
        END IF;

        temp := penalizari_ani(an);
        temp(luna) := temp(luna) + r.penalizari;
        penalizari_ani(an) := temp;

        IF r.penalizari > 0 THEN
            total_penalizari := total_penalizari + r.penalizari;
            numar_penalizari := numar_penalizari + 1;
        END IF;
    END;
END LOOP;

DBMS_OUTPUT.PUT_LINE('Numar penalizari: ' || numar_penalizari);
DBMS_OUTPUT.PUT_LINE('Suma penalizari: ' || total_penalizari);

FOR i IN 1 .. p_ani.COUNT LOOP
    IF penalizari_ani.EXISTS(p_ani(i)) THEN
        DBMS_OUTPUT.PUT_LINE('Anul ' || p_ani(i));
        temp := penalizari_ani(p_ani(i));
        FOR j IN 1 .. 12 LOOP
            DBMS_OUTPUT.PUT_LINE(' Luna ' || j || ': ' || temp(j));
        END LOOP;
    ELSE
        DBMS_OUTPUT.PUT_LINE('Nu exista penalizari in anul ' || p_ani(i));
    END IF;
END LOOP;

END;
/
BEGIN
    analiza_client(2, ani_nt(2022, 2023, 2024));
END;
/

```

The screenshot displays a database management interface with three main panels: Database Explorer, SQL Editor, and Services.

Database Explorer: Shows a tree view of the database structure. The 'STUDENT' schema is expanded, showing tables 'ACHIZITIE' and 'ANALIST'.

SQL Editor: Contains the following SQL code:

```
CREATE OR REPLACE PROCEDURE analiza_client(p_id_client NUMBER, p_anii_anizant  
TYPE venit_nt IS TABLE OF VENIT_CLIENT%ROWTYPE;  
venituri venit_nt;  
  
TYPE tipuri_t IS TABLE OF BOOLEAN INDEX BY VARCHAR2(100);  
tipuri tipuri_t;
```

Services: Displays the execution results of a query. The results are as follows:

Service	Time	Results
default	430 ms	Venit total: 8200
insert_into_tables	6	Venit maxim: 4000
insert_into_table		Tipuri de venit:
GARANT		Chirie
RATA	643 ms	PFA
RATA	643 ms	Salariu
ANALIST		Numar credite aprobate: 2
create_tables	1 s 6	Suma totala aprobata: 35000
create_tables		Numar penalizari: 6
6-subprogram_stoc		Suma penalizari: 1100
6-subprogram_s		Anul 2022
facultate		Luna 1: 0
EMPLOYEES		Luna 2: 0
EMPLOYEES		Luna 3: 0
JOBS		Luna 4: 0
EMPLOYEES2		Luna 5: 0
EMPLOYEES_ALME		Luna 6: 150
DEPARTMENTS		Luna 7: 0
ANGAJATI		Luna 8: 200
		Luna 9: 0
		Luna 10: 0
		Luna 11: 100
		Luna 12: 0
		Anul 2023

7 Cursoare - Subprogram Stocat Independent

7.1 Enunț

Banca dorește să analizeze nivelul de risc al clienților din punct de vedere al garanțiilor, ținând cont atât de venitul acestora, cât și de relația dintre client și garant.

În acest sens, se cere implementarea unui **subprogram stocat independent** care utilizează **două cursoare diferite**, dintre care **unul este parametrizat și dependent de celalalt**, pentru a realiza următoarea analiză:

1. Un **cursor neparametrizat** parcurge lista tuturor clientilor care au cel puțin un garant.
2. Un **cursor parametrizat**, dependent de primul, primește ca parametru **id_client** și parcurge toți garanții clientului.

Pentru fiecare client se vor determina și afișa următoarele informații:

- numărul total de garanți;
- venitul total al garanților;
- venitul maxim dintre garanți;
- numărul garanților considerați **valizi**;
- numărul garanților considerați **nevalizi**.

Un garant este considerat **nevalid** dacă relația acestuia cu clientul este una de rudenie:

- părinte (mamă, tată);
- copil;
- soț sau soție;
- frate sau soră.

Garanții care se află în una dintre relațiile de mai sus nu sunt luați în considerare pentru calculul nivelului de acoperire a clientului.

Pe baza garanților **valizi**, clientul este clasificat în funcție de nivelul de risc, astfel:

- *Risc scăzut* – există cel puțin un garant valid cu venit mai mare de 7000 RON;
- *Risc mediu* – există garanți valizi, dar cu venituri mai mici de 7000 RON;
- *Risc ridicat* – nu există garanți valizi.

7.2 Rezolvare

```
CREATE OR REPLACE PROCEDURE analiza_garanti AS
    cursor c_clienti is
        select distinct c.id_client, c.nume, c.prenume
        from client c
        join garant g on c.id_client = g.id_client;

    cursor c_garanti (my_id_client number) is
        select venit_garant, LOWER(relatie_client) as relatie
        from garant
        where id_client = my_id_client;

    my_nr_garanti NUMBER;
    my_nr_valizi NUMBER;
    my_nr_nevalizi NUMBER;
    my_venit_total NUMBER;
    my_venit_max NUMBER;
```

```
my_risc VARCHAR2(20);

begin
  for c in c_clienti loop
    my_nr_garanti := 0;
    my_nr_valizi := 0;
    my_nr_nevalizi := 0;
    my_venit_total := 0;
    my_venit_max := 0;

    for g in c_garanti(c.id_client) loop
      my_nr_garanti := my_nr_garanti + 1;

      if g.relatie not in ('sot', 'sotie', 'fiu', 'fiica', 'parinte') then
        my_nr_valizi := my_nr_valizi + 1;
        my_venit_total := my_venit_total + g.venit_garant;
        if g.venit_garant > my_venit_max then
          my_venit_max := g.venit_garant;
        end if;
      else
        my_nr_nevalizi := my_nr_nevalizi + 1;
      end if;
    end loop;

    if(my_nr_valizi = 0) then
      my_risc := 'Risc ridicat';
    else
      if(my_venit_max <= 7000) then
        my_risc := 'Risc mediu';
      else
        my_risc := 'Risc scazut';
      end if;
    end if;

    DBMS_OUTPUT.PUT_LINE('Client: ' || c.nume || ' ' || c.prenume);
    DBMS_OUTPUT.PUT_LINE('Numar garanti: ' || my_nr_garanti);
    DBMS_OUTPUT.PUT_LINE('Garanti valizi: ' || my_nr_valizi);
    DBMS_OUTPUT.PUT_LINE('Garanti nevalizi: ' || my_nr_nevalizi);
    DBMS_OUTPUT.PUT_LINE('Venit total garanti valizi: ' || my_venit_total);
    DBMS_OUTPUT.PUT_LINE('Venit maxim garant: ' || my_venit_max);
    DBMS_OUTPUT.PUT_LINE('Clasificare risc: ' || my_risc);
    DBMS_OUTPUT.PUT_LINE('-----');
  END LOOP;
END;
/
BEGIN
  analiza_garanti();
END;
/
```

The screenshot displays a database management interface with the following components:

- Database Explorer:** Shows a table named `STUDENT` with columns: `ID_CLIENT` (NUMBER), `NUME` (VARCHAR2(50)), `PRENUME` (VARCHAR), `ADRESA` (VARCHAR2), `TELEFON` (VARCHAR), `EMAIL` (VARCHAR2(10)), `CNP` (VARCHAR2(13)), and `RELATIE_CLIENT` (VARCHAR).
- Query Editor:** Contains a SQL query with a cursor named `7-sub_stocat_independent_cursore.sql`. The query is currently at the `END;` statement.
- Services:** A list of database services including `default`, `insert_into_tables`, `insert_into_table`, `7-sub_stocat_indep`, `GARANT`, `RATA`, `ANALIST`, `create_tables`, `6-subprogram_stoc`, `6-subprogram_s`, `facultate`, `EMPLOYEES`, `JOBS`, `EMPLOYEES2`, `EMPLOYEES_ALME`, and `DEPARTMENTS`.
- Output:** Displays the results of the query execution for the `STUDENT.CLIENT` table. The output shows the following data for the first two clients:

Client	Numar garantii	Garanti valizi	Garanti nevalizi	Venit total garanti valizi	Venit maxim garant	Clasificare risc
Popescu Andrei	4	3	1	21500	9800	Risc scazut
Ionescu Maria	3	3	0	14400	7000	Risc mediu

The output also includes a section for the `facultate` service, showing data for `EMPLOYEES`, `JOBS`, `EMPLOYEES2`, and `EMPLOYEES_ALME`.

8 NO_DATA_FOUND si TOO_MANY_ROWS - Subprogram Stocat Independent

8.1 Enunț

Pentru un credit identificat prin **id_credit**, banca dorește să determine tipul achiziției finanțate prin acel credit.

Se cere implementarea unui **subprogram stocat independent de tip funcție** care returnează **tipul achiziției**, utilizând o singură comandă SQL ce implică **exact trei tabele** din baza de date, și anume: **CREDIT, EVALUARE și ACHIZITIE**.

Funcția trebuie să trateze excepțiile:

- **NO_DATA_FOUND**, dacă nu există nicio achiziție
- **TOO_MANY_ROWS**, dacă există mai multe achiziții
- orice altă excepție.

8.2 Rezolvare

```
CREATE OR REPLACE FUNCTION get_tip_achizitie_credit (
    p_id_credit NUMBER
) RETURN VARCHAR2
IS
    v_tip_achizitie ACHIZITIE.tip_achizitie%TYPE;
BEGIN
    SELECT a.tip_achizitie
    INTO v_tip_achizitie
    FROM credit c
    JOIN evaluare e ON c.id_credit = e.id_credit
    JOIN achizitie a ON e.id_achizitie = a.id_achizitie
    WHERE c.id_credit = p_id_credit;

    RETURN v_tip_achizitie;

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RAISE_APPLICATION_ERROR(-20001,
            'Nu exista nicio achizitie asociata creditului dat');

    WHEN TOO_MANY_ROWS THEN
        RAISE_APPLICATION_ERROR(-20002,
            'Creditul are mai multe achizitii');

    WHEN OTHERS THEN
        RAISE_APPLICATION_ERROR(-20003,
            'Eroare neprevazuta');
END;
/
BEGIN
    DBMS_OUTPUT.PUT_LINE(get_tip_achizitie_credit(2));
END;
/
BEGIN
    DBMS_OUTPUT.PUT_LINE(get_tip_achizitie_credit(1));
END;
/
BEGIN
    DBMS_OUTPUT.PUT_LINE(get_tip_achizitie_credit(999));
END;
```

/

The screenshot displays the Oracle SQL Developer environment. The top pane shows a PL/SQL script with the following code:

```
34 BEGIN
35 DBMS_OUTPUT.PUT_LINE('A get_tip_achizitie_credit( p_id_crediti 1));
36 END;
```

The bottom pane shows the execution results, including a list of services and their execution times. The services listed are:

- 7-sub_stocaLinc
- GARANT 887 ms
- RATA 643 ms
- ANALIST
- create_tables 1 s 61
- 8-NDF_TMR_sub_st
- 6-subprogram_stoc
- 6-subprogram_s
- facultate
- EMPLOYEES
- EMPLOYEES
- JOBS
- EMPLOYEES2
- EMPLOYEES_ALME
- DEPARTMENTS
- ANGAJATI

The execution results pane shows the following output:

```
END;
[2025-12-19 15:09:16] completed in 68 ms
STUDENT> BEGIN
DBMS_OUTPUT.PUT_LINE(get_tip_achizitie_credit(2));
END;
[2025-12-19 15:09:21] completed in 23 ms
Masina
STUDENT> BEGIN
DBMS_OUTPUT.PUT_LINE(get_tip_achizitie_credit(1));
END;
[2025-12-19 15:09:23] [72000][20002]
[2025-12-19 15:09:23] ORA-20002: Creditul are mai multe achizitii
[2025-12-19 15:09:23] ORA-06512: at "STUDENT.GET_TIP_ACHIZITIE_CREDIT", line 22
[2025-12-19 15:09:23] ORA-06512: at line 2
[2025-12-19 15:09:23] Position: 0
STUDENT> BEGIN
DBMS_OUTPUT.PUT_LINE(get_tip_achizitie_credit(999));
END;
[2025-12-19 15:09:26] [72000][20001]
[2025-12-19 15:09:26] ORA-20001: Nu exista nicio achizitie asociata creditului dat
[2025-12-19 15:09:26] ORA-06512: at "STUDENT.GET_TIP_ACHIZITIE_CREDIT", line 18
[2025-12-19 15:09:26] ORA-06512: at line 2
[2025-12-19 15:09:26] Position: 0
```

The right pane shows a list of files, including:

- Probleme.sql
- Screenshot 2025-10-10 at 6.00.27 PM.png
- Laborator2
- IntroducereInPLSQL.sql
- Laborator3
- TipuriDeDate.sql
- Laborator5
- cursor.sql
- exercitiul1.sql
- exercitiul2.sql

9 Subprogram Stocat Independent Procedură - 2 Excepții Proprii

9.1 Enunț

Banca dorește să valideze acordarea unui credit pentru un client, pe baza veniturilor acestuia, a gradului de îndatorare și a deciziei rezultate în urma evaluării creditului.

Pentru un client identificat prin `id_client` și un credit identificat prin `id_credit`, se cunosc următoarele informații:

- veniturile lunare ale clientului, stocate în tabela **VENIT** și asociate clientului prin tabela **VENIT_CLIENT**;
- gradul de îndatorare al clientului, stocat în tabela **CLIENT**;
- suma creditului solicitat, stocată în tabela **CREDIT**;
- decizia finală a evaluării creditului, stocată în tabela **EVALUARE**.

Acordarea creditului este considerată validă dacă sunt îndeplinite simultan următoarele condiții:

1. venitul lunar total al clientului este cel puțin egal cu 5% din suma creditului;
2. decizia evaluării creditului este **APROBAT**;
3. gradul de îndatorare al clientului nu depășește valoarea de 40

Subprogramul stocat independent de tip procedură:

- primește ca parametri id-ul clientului și id-ul creditului;
- calculează venitul lunar total al clientului;
- verifică îndeplinirea condițiilor de acordare a creditului;
- tratează distinct următoarele situații:
 - venit insuficient;
 - credit neaprobat;
 - grad de îndatorare prea mare;
 - date inexistente sau inconsistente.

9.2 Rezolvare

```
CREATE OR REPLACE PROCEDURE validare_credit (p_id_client CLIENT.id_client%TYPE, p_id_credit CREDIT.
v_venit_total NUMBER;
v_grad_indatorare CLIENT.grad_indatorare%TYPE;
v_decizie EVALUARE.decizie%TYPE;
v_suma_credit CREDIT.SUMA_SOLICITATA%TYPE;
v_venit_minim NUMBER;

venit_insuficient EXCEPTION;
credit_neaprobat EXCEPTION;
grad_prea_mare EXCEPTION;
BEGIN
  SELECT SUM(vc.suma_lunara), c.grad_indatorare, e.decizie, cr.SUMA_SOLICITATA
  INTO v_venit_total, v_grad_indatorare, v_decizie, v_suma_credit
  FROM client c
  JOIN venit_client vc ON c.id_client = vc.id_client
  JOIN venit v ON vc.id_venit = v.id_venit
  JOIN credit cr ON cr.id_client = c.id_client
  JOIN evaluare e ON e.id_credit = cr.id_credit
  WHERE c.id_client = p_id_client AND cr.id_credit = p_id_credit
  GROUP BY c.grad_indatorare, e.decizie, cr.SUMA_SOLICITATA;
```

```
v_venit_minim := v_suma_credit * 0.05;

IF v_venit_total < v_venit_minim THEN
    RAISE venit_insuficient;
END IF;

IF LOWER(v_decizie) != 'aprobat' THEN
    RAISE credit_neaprobat;
END IF;

IF v_grad_indatorare > 40 THEN
    RAISE grad_prea_mare;
END IF;

DBMS_OUTPUT.PUT_LINE('Credit VALID');

EXCEPTION
    WHEN venit_insuficient THEN
        DBMS_OUTPUT.PUT_LINE('Venit insuficient: necesar minim = ' || v_venit_minim || ', venit total = ');
    WHEN credit_neaprobat THEN
        DBMS_OUTPUT.PUT_LINE('Creditul nu este aprobat');
    WHEN grad_prea_mare THEN
        DBMS_OUTPUT.PUT_LINE('Grad de indatorare prea mare');
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Creditul nu exista sau nu a fost evaluat');
    WHEN TOO_MANY_ROWS THEN
        DBMS_OUTPUT.PUT_LINE('Date inconsistente: mai multe evaluari gasite');
END;
/
--credit valid
BEGIN
    validare_credit(1, 1);
END;
/
--venit insuficient
BEGIN
    validare_credit(3, 9);
END;
/
--grad indatorare prea mare
BEGIN
    validare_credit(6, 10);
END;
/
--no data found
BEGIN
    validare_credit(999, 1);
END;
/
--too many rows
BEGIN
    validare_credit(1, 1);
END;
```

The screenshot displays a database management tool interface with the following components:

- Database Explorer:** Shows a tree view of the database structure, including a schema named 'STUDENT' with tables 'ACHIZITIE' and 'ANALIST'.
- SQL Editor:** Contains a script with the following SQL code:

```
--no data found
66
67 BEGIN
68     validate_credit(p_id_client 999, p_id_credit 1);
69 END;
70 /
71 --too many rows
72 BEGIN
73     validate_credit(p_id_client 1, p_id_credit 1);
74 END;
```
- Services:** A list of database services and their execution times, including 'EVALUARE', 'CREDIT', '9-2.exceptil_propri', 'insert_into_tables', 'insert_into_table', and 'CLIENT'.
- Output/Result:** Displays the execution results of the SQL script, showing messages such as '1 row retrieved starting from 1 in 528 ms', 'completed in 13 ms', and 'Creditul nu exista sau nu a fost evaluat'.

10 Trigger LMD

10.1 Enunt

Vrem corectitudinea datelor în procesul de evaluare a riscului, clasa de risc atribuită unui credit trebuie să fie în concordanță cu scorul de risc calculat.

Reguli de clasificare:

1. Clasa 'A' (risc scăzut): $scor_risc < 30$
2. Clasa 'B' (risc mediu): $30 \leq scor_risc < 50$
3. Clasa 'C' (risc ridicat): $scor_risc \geq 50$

Să se definească un *trigger* de tip LMD la nivel de comandă care:

- se activează la fiecare operație de inserare sau actualizare în tabelul EVALUARE;
- validează concordanța dintre câmpurile `scor_risc` și `clasa_risc` pentru toate înregistrările afectate;
- în cazul unei erori, generează o excepție folosind `RAISE_APPLICATION_ERROR`, afișând valorile incorecte și clasa de risc ce trebuia afișată conform scorului.

10.2 Rezolvare

```
CREATE OR REPLACE TRIGGER trg_validare_clasa_risc
AFTER INSERT OR UPDATE ON EVALUARE
DECLARE
    CURSOR c_evaluari IS
        SELECT id_evaluare, scor_risc, clasa_risc
        FROM EVALUARE;

    v_clasa VARCHAR2(10);
BEGIN
    FOR rec IN c_evaluari LOOP
        IF rec.scor_risc < 30 THEN
            v_clasa := 'A';
        ELSIF rec.scor_risc < 50 THEN
            v_clasa := 'B';
        ELSE
            v_clasa := 'C';
        END IF;

        IF rec.clasa_risc != v_clasa THEN
            RAISE_APPLICATION_ERROR(-20002,
                'Clasa risc inconsistentă pentru evaluarea ' || rec.id_evaluare ||
                ': scor=' || rec.scor_risc ||
                ', clasa=' || rec.clasa_risc ||
                ', așteptat=' || v_clasa);
        END IF;
    END LOOP;
END;
/
--declansare trigger
INSERT INTO EVALUARE (id_credit, id_analist, id_achizitie, tip_evaluare, scor_risc, clasa_risc, decizie)
VALUES (2, 1, 1, 'Revizuire', 55, 'A', 'APROBAT');
/
UPDATE EVALUARE
SET clasa_risc = 'B'
WHERE id_evaluare = 1 AND scor_risc = 25;
```

The screenshot displays the Oracle SQL Developer environment. The main editor shows the creation of a trigger named `trg_validare_clasa_risc`. The trigger is designed to validate the risk class based on the score and decision. It includes a loop to check for inconsistencies and a decision to either approve or reject the credit based on the score and decision.

```

1 CREATE OR REPLACE TRIGGER trg_validare_clasa_risc
2   AFTER INSERT ON EVALUARE
3   FOR EACH ROW
4   BEGIN
5     IF (scor_risc < 25 AND decizie = 'A') OR (scor_risc < 40 AND decizie = 'B') THEN
6       RAISE_APPLICATION_ERROR(-20002, 'Clasa risc inconsistenta pentru evaluarea 1: scor=' || scor_risc || ', clasa=' || clasa_risc || ', asteptat=' || v_clasa);
7     END IF;
8   END LOOP;
9 END;

```

The Services window shows the execution of the trigger. The trigger was executed successfully for the first time, but it failed for the second time due to an inconsistency in the risk class. The error messages are as follows:

- ORA-20002: Clasa risc inconsistenta pentru evaluarea 1: scor=25, clasa=B, asteptat=A
- ORA-06512: at "STUDENT.TRG_VALIDARE_CLASA_RISC", line 18
- ORA-06512: at "STUDENT.TRG_VALIDARE_CLASA_RISC", line 18
- ORA-04088: error during execution of trigger 'STUDENT.TRG_VALIDARE_CLASA_RISC' ...

The Database Explorer shows the schema structure, including the `EVALUARE` table and its columns: `ID_EVALUARE`, `ID_CREDIT`, `ID_ANALIST`, `ID_ACHIZITIE`, and `DATA_EVALUARI`.

11 Trigger LMD la nivel de linie

11.1 Enunt

În procesul de evaluare a achizițiilor, valoarea de evaluare stabilită de experți trebuie să respecte anumite limite pentru a asigura protecția băncii împotriva supraevaluărilor sau subevaluărilor.

Se impun următoarele condiții:

1. Valoarea de evaluare nu poate depăși valoarea de piață a bunului:

$$valoare_evaluare \leq valoare_piata$$

2. Valoarea de evaluare nu poate fi mai mică de 70% din valoarea de piață:

$$valoare_evaluare \geq 0.70 \times valoare_piata$$

11.2 Rezolvare

```
CREATE OR REPLACE TRIGGER trg_validare_evaluare
BEFORE INSERT OR UPDATE ON ACHIZITIE
FOR EACH ROW
DECLARE
    v_valoare_minima NUMBER;
BEGIN
    v_valoare_minima := :NEW.valoare_piata * 0.70;

    IF :NEW.valoare_evaluare > :NEW.valoare_piata THEN
        RAISE_APPLICATION_ERROR(-20003, 'Valoare evaluare (' || :NEW.valoare_evaluare ||
        ') nu poate depasi valoarea de piata (' || :NEW.valoare_piata || ')');
    END IF;

    IF :NEW.valoare_evaluare < v_valoare_minima THEN
        RAISE_APPLICATION_ERROR(-20004, 'Valoare evaluare (' || :NEW.valoare_evaluare ||
        ') este prea mica. Minim acceptat: ' || v_valoare_minima);
    END IF;
END;
/
INSERT INTO ACHIZITIE (tip_achizitie, procent_finantat, suma_avans, valoare_piata, valoare_evaluare,
data_evaluare, descriere)
VALUES ('Apartament', 80, 40000, 200000, 220000, SYSDATE, 'Evaluare supraevaluata');
/
INSERT INTO ACHIZITIE (tip_achizitie, procent_finantat, suma_avans, valoare_piata, valoare_evaluare,
data_evaluare, descriere)
VALUES ('Vila', 70, 60000, 300000, 180000, SYSDATE, 'Evaluare subevaluata');
/
UPDATE ACHIZITIE
SET valoare_evaluare = 160000
WHERE id_achizitie = 1;
```

The screenshot displays the Oracle SQL Developer environment. The top pane shows the Database Explorer with a tree view of the database schema, including tables like CLIENT, CREDIT, and EVALUARE. The middle pane shows the Services window, which lists various database services and their execution times. The bottom pane shows the SQL script editor, which contains a PL/SQL block with a trigger definition and several INSERT and UPDATE statements. The script is executed, and the output window shows the following error messages:

```
[72000][20003]
ORA-20003: Valoare evaluare (160000) nu poate depasi valoarea de piata (120000)
ORA-06512: at "STUDENT.TRG_VALIDARE_EVALUARE", line 7
ORA-04088: error during execution of trigger 'STUDENT.TRG_VALIDARE_EVALUARE'
Position: 7
```

The script also includes a trigger definition for 'STUDENT.TRG_VALIDARE_EVALUARE' and several INSERT and UPDATE statements for the 'ACHIZITIE' table. The output window shows the execution of these statements, including the error messages mentioned above.

12 Trigger LDD

12.1 Enunt

În sistemul bancar, este necesară monitorizarea tuturor modificărilor bazei de date.

Să se definească un *trigger* DDL care:

1. Se activează la operații **ALTER** sau **DROP** pe schema curentă;
2. Înregistrează în tabelul **LOG_DDL** informații despre:
 - tipul operației (**ALTER/DROP**);
 - numele și tipul obiectului modificat;
 - utilizatorul și timestamp-ul operației.
3. Afișează un mesaj dacă operația afectează unul din tabelele critice: **CLIENT**, **CREDIT**, **RATA**.

12.2 Rezolvare

```
CREATE TABLE LOG_DDL (
    id_log NUMBER GENERATED ALWAYS AS IDENTITY,
    tip_operatie VARCHAR2(20) NOT NULL,
    tip_obiect VARCHAR2(20),
    nume_obiect VARCHAR2(100) NOT NULL,
    utilizator VARCHAR2(50) NOT NULL,
    data_operatie TIMESTAMP DEFAULT SYSTIMESTAMP,

    CONSTRAINT pk_log_ddl PRIMARY KEY (id_log)
);

CREATE OR REPLACE TRIGGER trg_log_ddl
AFTER ALTER OR DROP ON SCHEMA
DECLARE
    v_tip_operatie VARCHAR2(20);
    v_tip_obiect VARCHAR2(20);
    v_nume_obiect VARCHAR2(100);
    PRAGMA AUTONOMOUS_TRANSACTION;
BEGIN
    v_tip_operatie := ORA_SYSEVENT();
    v_tip_obiect := ORA_DICT_OBJ_TYPE();
    v_nume_obiect := ORA_DICT_OBJ_NAME();

    INSERT INTO LOG_DDL (tip_operatie, tip_obiect, nume_obiect, utilizator )
    VALUES (v_tip_operatie,v_tip_obiect,v_nume_obiect,USER);

    COMMIT;
    IF v_tip_obiect = 'TABLE' AND
        v_nume_obiect IN ('CLIENT', 'CREDIT', 'RATA') THEN
        DBMS_OUTPUT.PUT_LINE('Avertizare! Tabel Critic! Refaceti daca ati facut o greseala!' || v_nume_
    END IF;
EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
END;
/
rollback;
ALTER TABLE ANALIST ADD test_col VARCHAR2(50);
ALTER TABLE CLIENT ADD test_col VARCHAR2(50);
ALTER TABLE ANALIST DROP COLUMN test_col;
```

```
ALTER TABLE CLIENT DROP COLUMN test_col;
```

```
SELECT tip_operatie, tip_obiect, nume_obiect, utilizator,  
       TO_CHAR(data_operatie, 'DD-MON-YYYY HH24:MI:SS') as data  
FROM LOG_DDL  
ORDER BY data_operatie DESC;
```

The screenshot shows the SQL Developer interface with the following components:

- Database Explorer:** Shows the schema structure for the @stud database, including tables like CLIENT, ANALIST, and GARANT.
- Script Editor:** Displays a series of SQL scripts (12-Trigger-LDD.sql, VENIT_CLIENT.sql, EVALUARE.sql, GAR.sql) that perform various ALTER and DROP operations on the database schema.
- Output Window:** Shows the execution results of the scripts, including messages like "ALTER TABLE ANALIST ADD test_col VARCHAR2(50);".
- Result Window:** Displays the output of the SELECT query, showing a table with columns: TIP_OPERATIE, TIP_OBIECT, NUME_OBIECT, UTILIZATOR, and DATA. The data is sorted by DATA in descending order.

TIP_OPERATIE	TIP_OBIECT	NUME_OBIECT	UTILIZATOR	DATA
ALTER	TABLE	CLIENT	STUDENT	07-JAN-2026 19:08:59
ALTER	TABLE	ANALIST	STUDENT	07-JAN-2026 19:08:56
ALTER	TABLE	CLIENT	STUDENT	07-JAN-2026 19:08:51
ALTER	TABLE	ANALIST	STUDENT	07-JAN-2026 19:08:46
ALTER	TABLE	CLIENT	STUDENT	07-JAN-2026 19:08:37
ALTER	TABLE	ANALIST	STUDENT	07-JAN-2026 19:08:23
ALTER	TABLE	CLIENT	STUDENT	07-JAN-2026 19:07:36
ALTER	TABLE	ANALIST	STUDENT	07-JAN-2026 19:07:16
DROP	TRIGGER	TRG_BACKUP_DDL	STUDENT	07-JAN-2026 19:06:36

The screenshot shows the SQL Developer interface with the following components:

- Database Explorer:** Shows the schema structure for the @stud database, including tables like CLIENT, ANALIST, and GARANT.
- Script Editor:** Displays a series of SQL scripts (12-Trigger-LDD.sql, VENIT_CLIENT.sql, EVALUARE.sql, GAR.sql) that perform various ALTER and DROP operations on the database schema.
- Output Window:** Shows the execution results of the scripts, including messages like "ALTER TABLE ANALIST ADD test_col VARCHAR2(50);".
- Result Window:** Displays the output of the SELECT query, showing a table with columns: TIP_OPERATIE, TIP_OBIECT, NUME_OBIECT, UTILIZATOR, and DATA. The data is sorted by DATA in descending order.

TIP_OPERATIE	TIP_OBIECT	NUME_OBIECT	UTILIZATOR	DATA
ALTER	TABLE	CLIENT	STUDENT	07-JAN-2026 19:08:59
ALTER	TABLE	ANALIST	STUDENT	07-JAN-2026 19:08:56
ALTER	TABLE	CLIENT	STUDENT	07-JAN-2026 19:08:51
ALTER	TABLE	ANALIST	STUDENT	07-JAN-2026 19:08:46
ALTER	TABLE	CLIENT	STUDENT	07-JAN-2026 19:08:37
ALTER	TABLE	ANALIST	STUDENT	07-JAN-2026 19:08:23
ALTER	TABLE	CLIENT	STUDENT	07-JAN-2026 19:07:36
ALTER	TABLE	ANALIST	STUDENT	07-JAN-2026 19:07:16
DROP	TRIGGER	TRG_BACKUP_DDL	STUDENT	07-JAN-2026 19:06:36

13 Pachet: Flux Actiuni Integrate

13.1 Enunt

Să se implementeze pachetul PACHET_PLATI care să conțină:

1. Tipuri de date complexe:

- tip_detalii_plata - informații complete despre o plată
- tip_restanta - detalii despre o rată restantă
- tip_lista_restante - colecție de rate restante

2. Funcții:

- calculeaza_penalizare - calculează penalizarea pentru o rată în funcție de zilele de întârziere
- verifica_restante_client - returnează numărul de rate restante pentru un client
- obtine_total_datorie - calculează datoria totală (rate + penalizări) pentru un credit

3. Proceduri:

- proceseaza_plata - procesează o plată și actualizează statusul ratei
- marcheaza_intarzieri - marchează automat toate ratele care au depășit scadența
- genereaza_raport_restante - generează raport cu toate restanțele active

13.2 Rezolvare

```
CREATE OR REPLACE PACKAGE PACHET_PLATI AS
  TYPE tip_detalii_plata IS RECORD (
    id_plata NUMBER,
    id_rata NUMBER,
    id_credit NUMBER,
    numar_rata NUMBER,
    suma_rata NUMBER,
    suma_platita NUMBER,
    suma_ramasa NUMBER,
    penalizare NUMBER,
    data_scadenta DATE,
    data_plata DATE,
    zile_intarziere NUMBER,
    status_rata VARCHAR2(20)
  );
  TYPE tip_restanta IS RECORD (
    id_client NUMBER,
    nume_client VARCHAR2(100),
    id_credit NUMBER,
    id_rata NUMBER,
    numar_rata NUMBER,
    suma_rata NUMBER,
    penalizare NUMBER,
    total_de_plata NUMBER,
    data_scadenta DATE,
    zile_intarziere NUMBER
  );
  TYPE tip_lista_restante IS TABLE OF tip_restanta;

  -- FUNCTII

  FUNCTION calculeaza_penalizare(p_id_rata IN NUMBER) RETURN NUMBER;
  FUNCTION verifica_restante_client(p_id_client IN NUMBER) RETURN NUMBER;
```

```
FUNCTION obtine_total_datorie(p_id_credit IN NUMBER) RETURN NUMBER;

-- PROCEDURI
PROCEDURE proceseaza_plata(
    p_id_rata IN NUMBER,
    p_suma_platita IN NUMBER,
    p_metoda_plata IN VARCHAR2 DEFAULT 'TRANSFER'
);
PROCEDURE marcheaza_intarzieri;
PROCEDURE genereaza_raport_restante;

END PACHET_PLATI;
/

CREATE OR REPLACE PACKAGE BODY PACHET_PLATI AS

    c_penalizare_zi NUMBER := 0.5;
    FUNCTION calculeaza_penalizare(
        p_id_rata IN NUMBER
    ) RETURN NUMBER IS
        v_suma_rata NUMBER;
        v_data_scadenta DATE;
        v_zile_intarziere NUMBER;
        v_penalizare NUMBER := 0;
    BEGIN
        SELECT suma_rata, data_scadenta
        INTO v_suma_rata, v_data_scadenta
        FROM RATA
        WHERE id_rata = p_id_rata;
        v_zile_intarziere := TRUNC(SYSDATE) - TRUNC(v_data_scadenta);
        IF v_zile_intarziere > 0 THEN
            v_penalizare := v_suma_rata*(c_penalizare_zi/100)*v_zile_intarziere;
        END IF;
        RETURN ROUND(v_penalizare, 2);
    END calculeaza_penalizare;
    FUNCTION verifica_restante_client(
        p_id_client IN NUMBER
    ) RETURN NUMBER IS
        v_numar_restante NUMBER := 0;
    BEGIN
        SELECT COUNT(*)
        INTO v_numar_restante
        FROM RATA r
        JOIN CREDIT c ON r.id_credit = c.id_credit
        WHERE c.id_client = p_id_client
        AND r.status_rata IN ('NEPLATITA', 'INTARZIATA')
        AND r.data_scadenta < SYSDATE;
        RETURN v_numar_restante;
    END verifica_restante_client;

    FUNCTION obtine_total_datorie(
        p_id_credit IN NUMBER
    ) RETURN NUMBER IS
        v_total_rate NUMBER := 0;
        v_total_penalizari NUMBER := 0;
        v_total_datorie NUMBER := 0;
    BEGIN
```

```
SELECT NVL(SUM(suma_rata), 0)
INTO v_total_rate
FROM RATA
WHERE id_credit = p_id_credit AND status_rata IN ('NEPLATITA', 'INTARZIATA');

SELECT NVL(SUM(penalizari), 0)
INTO v_total_penalizari
FROM RATA
WHERE id_credit = p_id_credit
AND status_rata IN ('NEPLATITA', 'INTARZIATA');

v_total_datorie := v_total_rate + v_total_penalizari;

RETURN ROUND(v_total_datorie, 2);

END obtine_total_datorie;

PROCEDURE proceseaza_plata(
    p_id_rata IN NUMBER,
    p_suma_platita IN NUMBER,
    p_metoda_plata IN VARCHAR2 DEFAULT 'TRANSFER'
) IS
    v_suma_rata NUMBER;
    v_penalizare NUMBER;
    v_total_de_plata NUMBER;
    v_status_nou VARCHAR2(20);
    v_suma_anterioara NUMBER := 0;
BEGIN
    SELECT suma_rata, penalizari
    INTO v_suma_rata, v_penalizare
    FROM RATA
    WHERE id_rata = p_id_rata;

    SELECT NVL(SUM(suma_platita), 0)
    INTO v_suma_anterioara
    FROM PLATA
    WHERE id_rata = p_id_rata;

    v_total_de_plata := v_suma_rata + v_penalizare;

    IF (v_suma_anterioara + p_suma_platita) >= v_total_de_plata THEN
        v_status_nou := 'PLATITA';
    ELSE
        v_status_nou := 'PARTIAL_PLATITA';
    END IF;

    INSERT INTO PLATA (id_rata, data_plata, suma_platita, metoda_plata)
    VALUES (p_id_rata, SYSDATE, p_suma_platita, p_metoda_plata);

    UPDATE RATA
    SET status_rata = v_status_nou
    WHERE id_rata = p_id_rata;

    COMMIT;

    DBMS_OUTPUT.PUT_LINE(' plata realizata cu succes!');
    DBMS_OUTPUT.PUT_LINE(' suma platita: ' || p_suma_platita || ' RON');
    DBMS_OUTPUT.PUT_LINE(' status nou: ' || v_status_nou);
```

```
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('eroare: rata nu exista!');
        ROLLBACK;
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('eroare procesare plata: ');
        ROLLBACK;
END proceseaza_plata;

PROCEDURE marcheaza_intarzieri IS
    v_count NUMBER := 0;
    v_penalizare NUMBER;
BEGIN
    FOR rec IN (
        SELECT id_rata
        FROM RATA
        WHERE status_rata = 'NEPLATITA'
        AND data_scadenta < SYSDATE
    ) LOOP
        v_penalizare := calculeaza_penalizare(rec.id_rata);

        UPDATE RATA
        SET status_rata = 'INTARZIATA',
            penalizari = v_penalizare
        WHERE id_rata = rec.id_rata;

        v_count := v_count + 1;
    END LOOP;

    COMMIT;
    DBMS_OUTPUT.PUT_LINE('rate marcate' || v_count);
    DBMS_OUTPUT.PUT_LINE('data: ' || TO_CHAR(SYSDATE, 'DD-MON-YYYY HH24:MI:SS'));

END marcheaza_intarzieri;

PROCEDURE genereaza_raport_restante IS
    v_restante tip_lista_restante;
    v_total_general NUMBER := 0;
BEGIN
    SELECT
        c.id_client,
        cl.numa || ' ' || cl.prenume,
        c.id_credit,
        r.id_rata,
        r.numar_rata,
        r.suma_rata,
        r.penalizari,
        r.suma_rata + r.penalizari,
        r.data_scadenta,
        TRUNC(SYSDATE) - TRUNC(r.data_scadenta)
    BULK COLLECT INTO v_restante
    FROM RATA r
    JOIN CREDIT c ON r.id_credit = c.id_credit
    JOIN CLIENT cl ON c.id_client = cl.id_client
    WHERE r.status_rata IN ('NEPLATITA', 'INTARZIATA')
    AND r.data_scadenta < SYSDATE
    ORDER BY r.data_scadenta;
```

```

DBMS_OUTPUT.PUT_LINE('RAPORT RESTANTE ACTIVE');
DBMS_OUTPUT.PUT_LINE('data: ' || TO_CHAR(SYSDATE, 'DD-MON-YYYY'));
DBMS_OUTPUT.PUT_LINE('');

IF v_restante.COUNT = 0 THEN
    DBMS_OUTPUT.PUT_LINE('nu exista restante active.');
```

ELSE

```

    FOR i IN 1..v_restante.COUNT LOOP
        DBMS_OUTPUT.PUT_LINE('client: ' || v_restante(i).nume_client);
        DBMS_OUTPUT.PUT_LINE(' credit ID: ' || v_restante(i).id_credit || ' Rata: ' || v_restante(i).rata);
        DBMS_OUTPUT.PUT_LINE(' suma rata: ' || v_restante(i).suma_rata || ' RON');
        DBMS_OUTPUT.PUT_LINE(' penalizare: ' || v_restante(i).penalizare || ' RON');
        DBMS_OUTPUT.PUT_LINE(' TOTAL DE PLATA: ' || v_restante(i).total_de_plata || ' RON');
        DBMS_OUTPUT.PUT_LINE(' scadenta: ' || TO_CHAR(v_restante(i).data_scadenta, 'DD-MON-YYYY'));
        DBMS_OUTPUT.PUT_LINE(' zile intarziere: ' || v_restante(i).zile_intarziere);
        DBMS_OUTPUT.PUT_LINE('-----');
```

 v_total_general := v_total_general + v_restante(i).total_de_plata;

```

    END LOOP;

    DBMS_OUTPUT.PUT_LINE('total restante: ' || v_restante.COUNT);
    DBMS_OUTPUT.PUT_LINE('suma totala de incasat: ' || ROUND(v_total_general, 2) || ' RON');
END IF;

EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('eroare generare raport: ' || SQLERRM);
END genereaza_raport_restante;

END PACHET_PLATI;
/

BEGIN
    PACHET_PLATI.marcheaza_intarzieri;
END;
/

DECLARE
    v_restante NUMBER;
BEGIN
    --introducem orice client
    v_restante := PACHET_PLATI.verifica_restante_client(1);
    DBMS_OUTPUT.PUT_LINE('clientul 1 are ' || v_restante || ' rate restante.');
```

END;

/

```

DECLARE
    v_datorie NUMBER;
BEGIN
    --introducem orice client
    v_datorie := PACHET_PLATI.obtine_total_datorie(1);
    DBMS_OUTPUT.PUT_LINE('datoria totala pentru creditul 1: ' || v_datorie || ' RON');
```

END;

/

```

BEGIN
    PACHET_PLATI.proceseaza_plata(1, 1500, 'CARD');
```

END;

/

```
BEGIN
    PACHET_PLATI.genereaza_raport_restante;
END;
/
```

The screenshot displays the Oracle SQL Developer environment. The top pane shows the Database Explorer with a tree view of the schema, including tables like PACHET_PLATI, VENIT_CLIENT, and GARANT. The bottom pane shows the SQL Editor with a PL/SQL script. The script defines a procedure PROCESAZA_PLATA that calculates the total due amount (datorie) for a client, considering interest (rate) and the principal (suma). The script is executed, and the output shows the results of the procedure call.

```

STUDENT> BEGIN
    PACHET_PLATI.marcheaza_intarzieri;
END;
[2026-01-07 23:02:41] completed in 14 ms
MARCARÉ INTARZIERI COMPLETATA
Rate marcate ca intarzieri: 0
Data procesarii: 07-JAN-2026 21:02:41
STUDENT> DECLARE
    v_restante NUMBER;
BEGIN
    v_restante := PACHET_PLATI.verifica_restante_client(1);
    DBMS_OUTPUT.PUT_LINE('Clientul 1 are ' || v_restante || ' rate restante.');
```